UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA

LICENCIATURA DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN Y CIENCIAS DE LA COMPUTACIÓN

SEDE: LICEO DEL MONTE 7691 **JORNADA:** DOMINGO **SECCIÓN:** “C”



ALGORITMOS

PROYECTO INTEGRADOR: SISTEMA DE INVENTARIO Y VENTAS

ESTUDIANTE:

FABRICIO JAIR RIVAS VELÁSQUEZ

CARNÉ:

7691-26-21930

DOCENTE:

Ing. Luis Fernando Alvarado Cruz

GUATEMALA, 11 DE OCTUBRE DE 2026

Patrick's VideoGames

Durante el desarrollo de mi proyecto Patrick's VideoGames, aprendí cómo utilizar Python para crear un sistema que permita llevar el control del inventario y las ventas de una tienda de videojuegos. Para organizar la información, utilicé cadenas de texto para manejar datos como los nombres de los productos, números para controlar los precios y las existencias, y archivos CSV para guardar la información y evitar que se pierda al cerrar el programa.

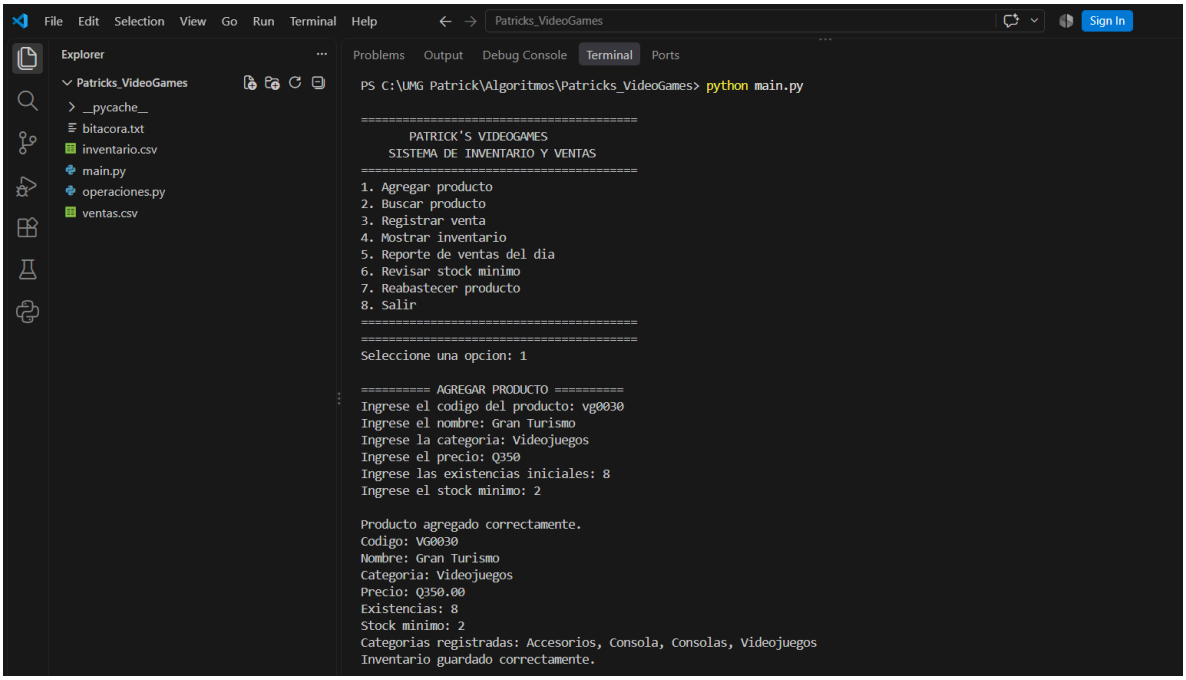
Una de las partes que más se me dificultó fue controlar las existencias al momento de realizar una venta, porque debía asegurarme de que el producto existiera y que hubiera suficientes unidades disponibles. Para resolverlo, trabajé en las validaciones del sistema, de manera que no se pudiera vender un producto que no estuviera registrado ni una cantidad mayor a la disponible. Esto me ayudó a comprender que no basta con hacer que un programa funcione, sino que también debe estar preparado para manejar errores.

Durante el desarrollo utilicé ChatGPT como apoyo para resolver dudas, comprender mejor algunos conceptos de Python y buscar soluciones para ciertas partes del código. Sin embargo, no se trató solamente de copiar las respuestas, sino de revisar las sugerencias, adaptarlas a las necesidades de mi proyecto y comprobar que funcionaran correctamente. De esta manera, pude aprender más sobre el funcionamiento del sistema y mejorar mi comprensión de la programación.

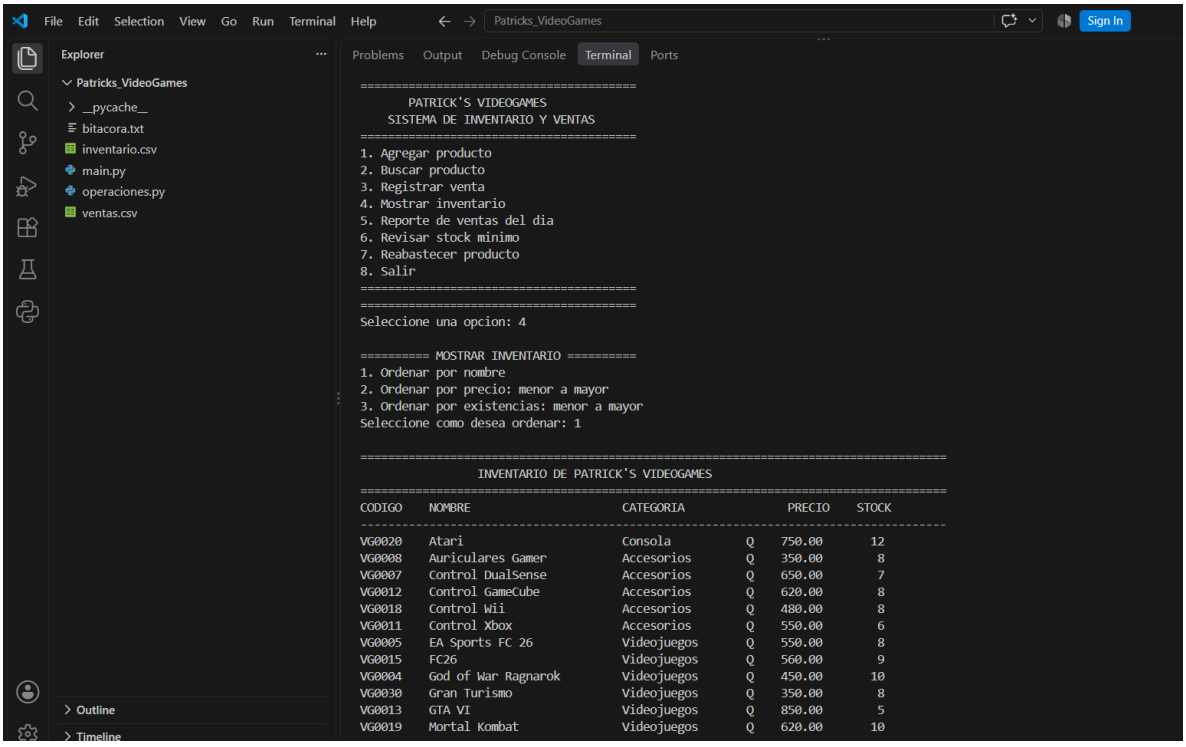
En conclusión, este proyecto me permitió poner en práctica los conocimientos adquiridos en clase y enfrentar dificultades que me ayudaron a aprender. También comprendí la importancia de organizar correctamente los datos, validar las operaciones y guardar la información de forma permanente. Considero que Patrick's VideoGames representa una oportunidad para seguir mejorando mis habilidades como estudiante de Ingeniería en Sistemas.

PRUEBAS DE USO

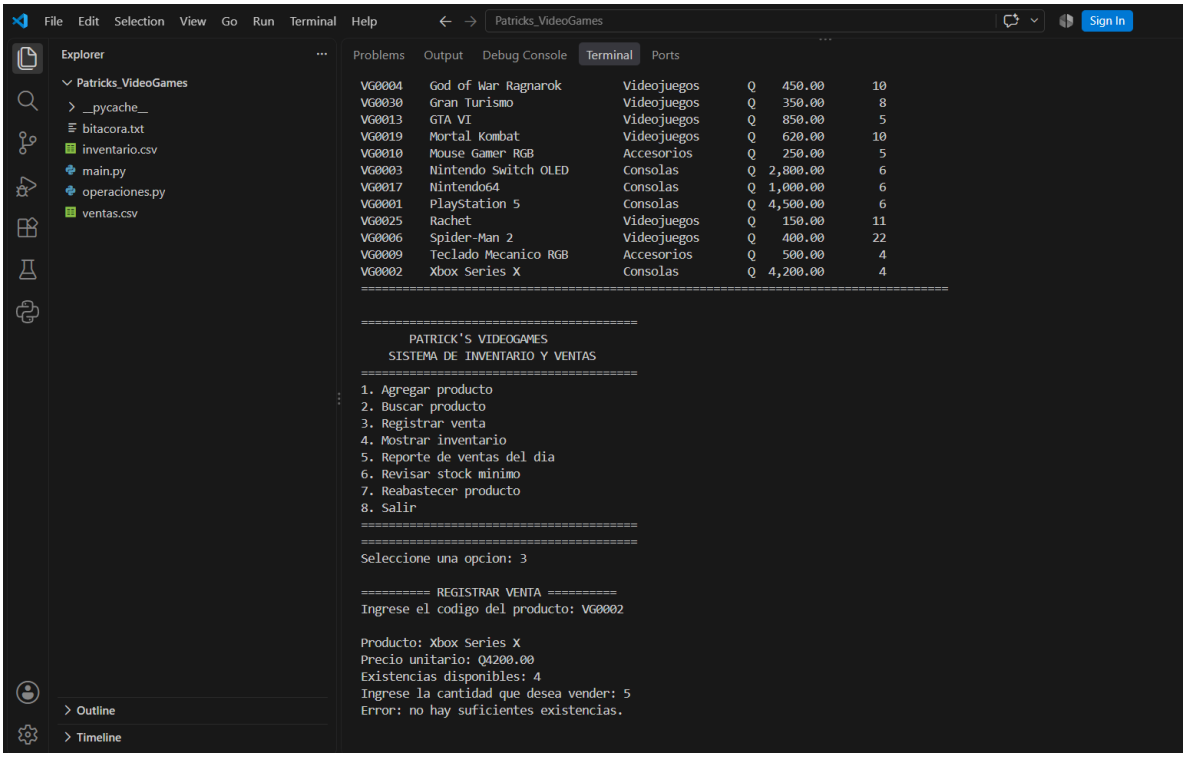
REGISTRAR UN PRODUCTO:



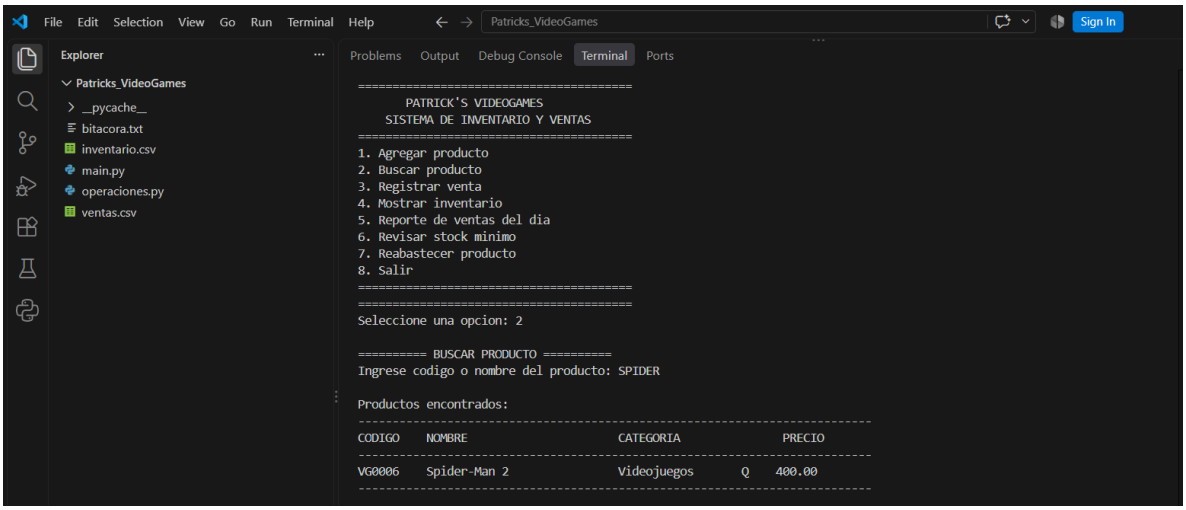
CONSULTA DE INVENTARIO Y REGISTRO DE NUEVO PRODUCTO:



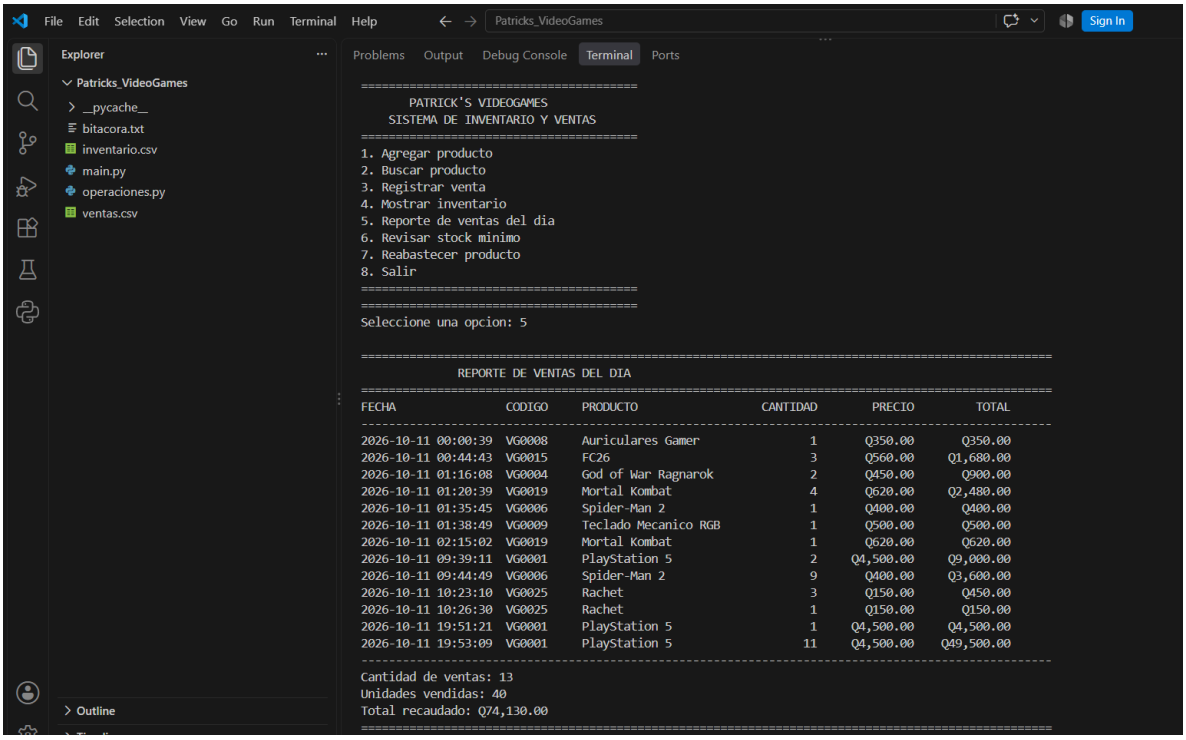
VENTA CON EXISTENCIAS INSUFICIENTES:



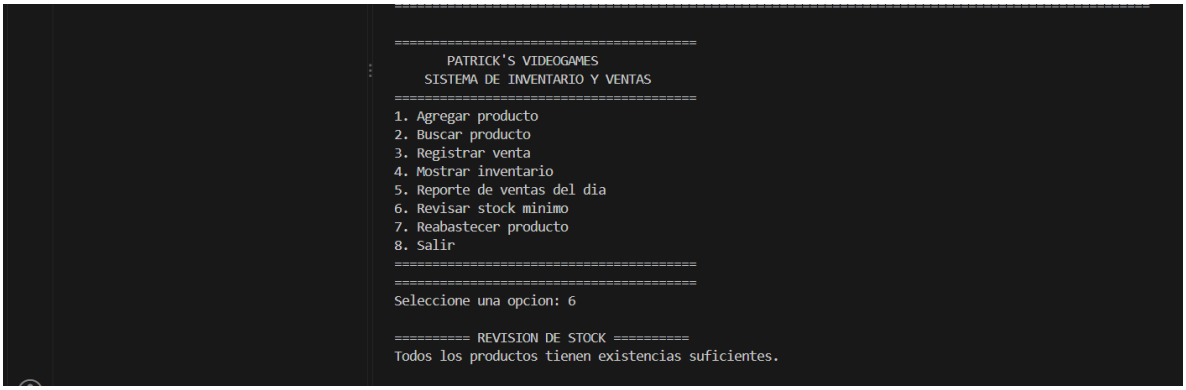
BUSCAR UN PRODUCTO:



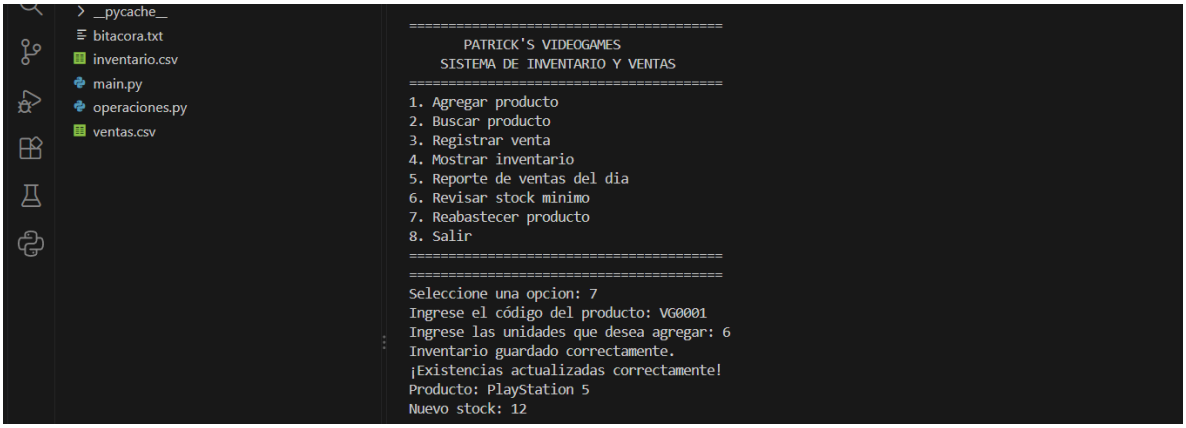
REPORTE DE VENTAS DEL DIA:



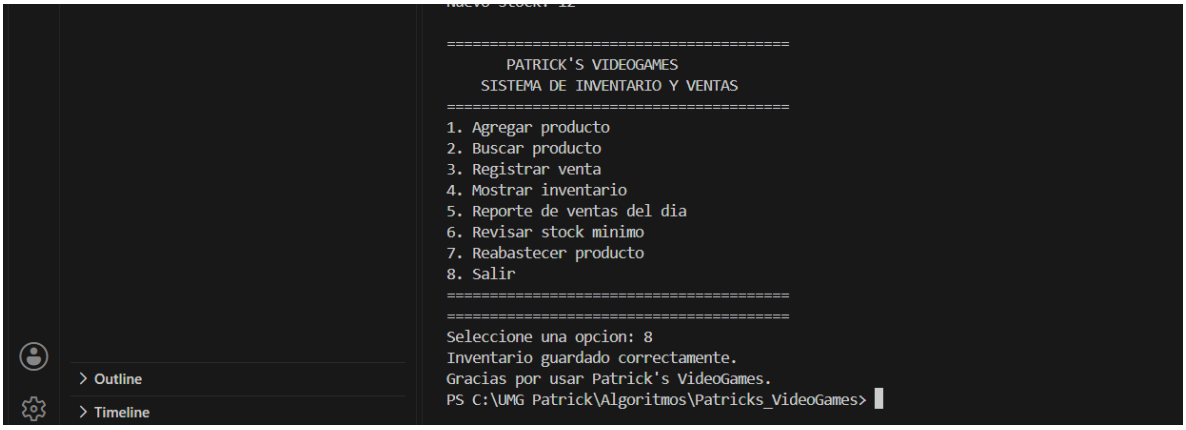
REGLA: STOCK MINIMO



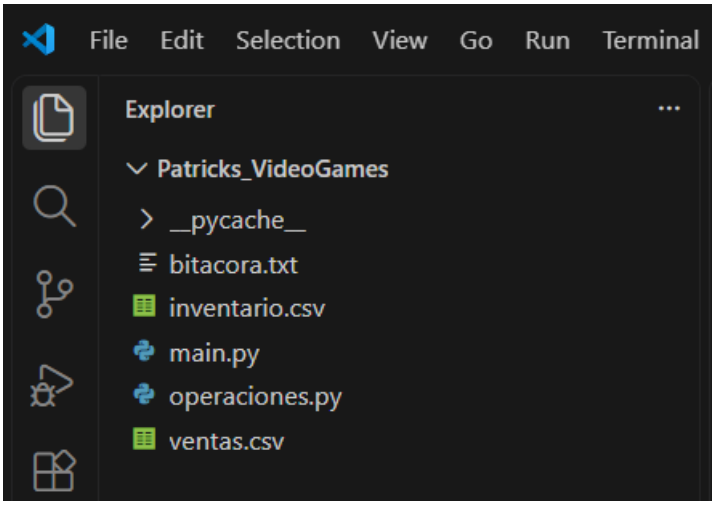
REABASTECER UN PRODUCTO:



GUARDAR Y SALIR:



ARCHIVOS Y MODULOS UTILIZADOS:



CODIGO UTILIZADO:

```
import csv

from pathlib import Path

from datetime import datetime

from operaciones import validar_codigo


ARCHIVO_INVENTARIO = Path(__file__).resolve().parent / "inventario.csv"


ARCHIVO_VENTAS = Path(__file__).resolve().parent / "ventas.csv"


ARCHIVO_BITACORA = Path(__file__).resolve().parent / "bitacora.txt"


ventas = []


def cargar_inventario():
  """
  Carga los productos desde el archivo CSV.

  Parámetros:
    Ninguno.

  Retorna:
    list: Lista de diccionarios con los productos cargados.
    None: Si ocurre un error que impide cargar el inventario.
  """

  inventario = []

  try:
    with open(ARCHIVO_INVENTARIO, "r", newline="", encoding="utf-8") as archivo:
      lector = csv.DictReader(archivo)

      campos = [
        "codigo",
        "nombre",
        "categoria",
        "precio",
        "stock",
        "stock_minimo"
```

```
            ]

            if lector.fieldnames is None or not all(
                campo in lector.fieldnames for campo in campos
            ):
                print("Error: el archivo de inventario no tiene los encabezados correctos.")
                return None

            for fila in lector:
                producto = {
                    "codigo": fila["codigo"],
                    "nombre": fila["nombre"],
                    "categoria": fila["categoria"],
                    "precio": float(fila["precio"]),
                    "stock": int(fila["stock"]),
                    "stock_minimo": int(fila["stock_minimo"])
                }

                inventario.append(producto)

    except FileNotFoundError:
        print("No se encontro el archivo de inventario.")
        guardar_inventario([])
        return []

    except (ValueError, KeyError) as error:
        print(f"Error en los datos del inventario: {error}")
        return None

    except OSError as error:
        print(f"No se pudo leer el archivo: {error}")
        return None

    return inventario

def guardar_inventario(inventario):
    """
    Guarda los productos del inventario en un archivo CSV.

    Parámetros:
```

```
    inventario (list): Lista de diccionarios con los datos
    de los productos.

  Retorna:
    None: Guarda la información y muestra el resultado.
  """

  try:
    with open(ARCHIVO_INVENTARIO, "w", newline="", encoding="utf-8") as archivo:

      campos = [
        "codigo",
        "nombre",
        "categoria",
        "precio",
        "stock",
        "stock_minimo"
      ]

      escritor = csv.DictWriter(archivo, fieldnames=campos)

      escritor.writeheader()

      for producto in inventario:
        escritor.writerow(producto)

    print("Inventario guardado correctamente.")

  except OSError as error:
    print(f"Error al guardar el inventario: {error}")

def agregar_producto(inventario):
  """
  Solicita los datos de un producto y lo agrega al inventario
  después de validar la información ingresada.

  Parámetros:
    inventario (list): Lista de diccionarios donde se almacenan
    los productos registrados.
```

```
Retorna:
    None: Agrega el producto si los datos son válidos.
"""

print("\n========== AGREGAR PRODUCTO ==========")

codigo = input("Ingrese el codigo del producto: ").strip().upper()

if not validar_codigo(codigo):
    print("Codigo invalido. Ejemplo correcto: VG0011")
    return

for producto in inventario:
    if producto["codigo"] == codigo:
        print("Error: ya existe un producto con ese codigo.")
        return

nombre = " ".join(input("Ingrese el nombre: ").strip().split())
categoria = " ".join(input("Ingrese la categoria: ").strip().split()).title()

if not nombre or not categoria:
    print("El nombre y la categoria son obligatorios.")
    return

try:
    precio = float(input("Ingrese el precio: Q").strip().replace(",", "."))

    if precio <= 0:
        print("El precio debe ser mayor que cero.")
        return

    stock = int(input("Ingrese las existencias iniciales: "))

    if stock < 0:
        print("Las existencias no pueden ser negativas.")
        return

    stock_minimo = int(input("Ingrese el stock minimo: "))

    if stock_minimo < 0:
```

```
            print("El stock minimo no puede ser negativo.")
            return

        except ValueError:
            print("Error: ingrese valores numericos validos.")
            return

        producto = {
            "codigo": codigo,
            "nombre": nombre,
            "categoria": categoria,
            "precio": round(precio, 2),
            "stock": stock,
            "stock_minimo": stock_minimo
        }

        inventario.append(producto)

        print("\nProducto agregado correctamente.")
        print(f"Codigo: {producto['codigo']}")
        print(f"Nombre: {producto['nombre']}")
        print(f"Categoria: {producto['categoria']}")
        print(f"Precio: Q{producto['precio']:.2f}")
        print(f"Existencias: {producto['stock']}")
        print(f"Stock minimo: {producto['stock_minimo']}")

        categorias = {p["categoria"] for p in inventario}
        print(f"Categorias registradas: {', '.join(sorted(categorias))}")
        registrar_bitacora(
            f"Producto agregado: {producto['codigo']} - {producto['nombre']}"
        )

    def buscar_producto(inventario):
        """
        Busca productos por código o por coincidencia parcial del nombre.

        Parámetros:
            inventario (list): Lista de diccionarios con los productos.

        Retorna:
```

```
    None: Muestra los productos que coinciden con la búsqueda.
"""

print("\n========== BUSCAR PRODUCTO ==========")

termino = input("Ingrese codigo o nombre del producto: ").strip()

if not termino:
    print("Debe ingresar un codigo o nombre para buscar.")
    return

termino = termino.casefold()

encontrados = []

for producto in inventario:
    codigo = producto["codigo"].casefold()
    nombre = producto["nombre"].casefold()

    if termino in codigo or termino in nombre:
        encontrados.append(producto)

if len(encontrados) == 0:
    print("No se encontraron productos.")
    return

print("\nProductos encontrados:")
print("-" * 75)

print(
    f"{'CODIGO':<10}"
    f"{'NOMBRE':<28}"
    f"{'CATEGORIA':<18}"
    f"{'PRECIO':>12}"
)

print("-" * 75)

for producto in encontrados:
    print(
```

```
            f"{producto['codigo']:<10}"
            f"{producto['nombre']:<28}"
            f"{producto['categoria']:<18}"
            f"Q{producto['precio']:>10,.2f}"
        )

    print("-" * 75)


from datetime import datetime


def guardar_venta(codigo, nombre, cantidad, precio_unitario, total):
    """
    Guarda los datos de una venta en el archivo CSV.

    Parámetros:
        codigo (str): Código del producto vendido.
        nombre (str): Nombre del producto.
        cantidad (int): Cantidad de unidades vendidas.
        precio_unitario (float): Precio de cada unidad.
        total (float): Importe total de la venta.

    Retorna:
        None: Guarda la venta y muestra el resultado.
    """

    try:
        archivo_nuevo = (
    not ARCHIVO_VENTAS.exists()
    or ARCHIVO_VENTAS.stat().st_size == 0
    )

        with open(ARCHIVO_VENTAS, "a", newline="", encoding="utf-8") as archivo:
            campos = [
                "fecha",
                "codigo",
                "nombre",
                "cantidad",
                "precio_unitario",
                "total"
            ]
```

```
            escritor = csv.DictWriter(archivo, fieldnames=campos)

            if archivo_nuevo:
                escritor.writeheader()

            escritor.writerow({
                "fecha": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
                "codigo": codigo,
                "nombre": nombre,
                "cantidad": cantidad,
                "precio_unitario": precio_unitario,
                "total": total
            })

        print("Venta guardada en el historial.")

    except OSError as error:
        print(f"Error al guardar la venta: {error}")


def registrar_venta(inventario):
    """
    Registra una venta, valida las existencias y actualiza el inventario.

    Parámetros:
        inventario (list): Lista de diccionarios con los productos.

    Retorna:
        None: Registra la venta si los datos son válidos y hay existencias.
    """

    print("\n========== REGISTRAR VENTA ==========")

    codigo = input("Ingrese el codigo del producto: ").strip().upper()

    if not validar_codigo(codigo):
        print("Codigo invalido. Ejemplo correcto: VG0001")
        return

    producto_encontrado = None
```

```
for producto in inventario:
    if producto["codigo"] == codigo:
        producto_encontrado = producto
        break

if producto_encontrado is None:
    print("Error: el producto no existe.")
    return

print(f"\nProducto: {producto_encontrado['nombre']}")
print(f"Precio unitario: Q{producto_encontrado['precio']:.2f}")
print(f"Existencias disponibles: {producto_encontrado['stock']}")

try:
    cantidad = int(input("Ingrese la cantidad que desea vender: "))

    if cantidad <= 0:
        print("La cantidad debe ser mayor que cero.")
        return

    if cantidad > producto_encontrado["stock"]:
        print("Error: no hay suficientes existencias.")
        return

except ValueError:
    print("Error: debe ingresar una cantidad numerica valida.")
    return

total = producto_encontrado["precio"] * cantidad
venta = (
    datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
    producto_encontrado["codigo"],
    producto_encontrado["nombre"],
    cantidad,
    producto_encontrado["precio"],
    total
)

ventas.append(venta)
```

```
    producto_encontrado["stock"] -= cantidad

    print("\n========== VENTA REGISTRADA ==========")
    print(f"Producto: {producto_encontrado['nombre']}")
    print(f"Cantidad vendida: {cantidad}")
    print(f"Precio unitario: Q{producto_encontrado['precio']:.2f}")
    print(f"Total de la venta: Q{total:.2f}")
    print(f"Existencias restantes: {producto_encontrado['stock']}")

    guardar_venta(
        producto_encontrado["codigo"],
        producto_encontrado["nombre"],
        cantidad,
        producto_encontrado["precio"],
        total
    )

    guardar_inventario(inventario)

    registrar_bitacora(
        f"Venta registrada: {producto_encontrado['codigo']} - "
        f"{producto_encontrado['nombre']} - "
        f"Cantidad: {cantidad} - Total: Q{total:.2f}"
    )

def reporte_ventas_dia():
    """
    Genera un reporte de las ventas realizadas durante el día actual.

    Parámetros:
        Ninguno.

    Retorna:
        None: Muestra las ventas, las unidades vendidas y el total recaudado.
    """

    print("\n" + "=" * 100)
    print("           REPORTE DE VENTAS DEL DIA")
    print("=" * 100)
```

```
fecha_actual = datetime.now().strftime("%Y-%m-%d")

total_recaudado = 0
unidades_vendidas = 0
cantidad_ventas = 0

try:
    with open(ARCHIVO_VENTAS, "r", newline="", encoding="utf-8-sig") as archivo:
        lector = csv.DictReader(archivo)

        print(
            f"{'FECHA':<20} "
            f"{'CODIGO':<10} "
            f"{'PRODUCTO':<25} "
            f"{'CANTIDAD':>8} "
            f"{'PRECIO':>13} "
            f"{'TOTAL':>13}"
        )

        print("-" * 100)

        for venta in lector:
            if venta["fecha"].startswith(fecha_actual):

                cantidad = int(venta["cantidad"])
                precio = float(venta["precio_unitario"])
                total = float(venta["total"])

                print(
                    f"{venta['fecha']:<20} "
                    f"{venta['codigo']:<10} "
                    f"{venta['nombre']:<25} "
                    f"{cantidad:>8} "
                    f"{('Q' + format(precio, ',.2f')):>13} "
                    f"{('Q' + format(total, ',.2f')):>13}"
                )

                total_recaudado += total
                unidades_vendidas += cantidad
```

```
                cantidad_ventas += 1

        print("-" * 100)

        if cantidad_ventas == 0:
            print("No hay ventas registradas durante el dia de hoy.")
        else:
            print(f"Cantidad de ventas: {cantidad_ventas}")
            print(f"Unidades vendidas: {unidades_vendidas}")
            print(f"Total recaudado: Q{total_recaudado:,.2f}")

        print("=" * 100)

    except FileNotFoundError:
        print("Todavia no existe un archivo de ventas.")

    except (ValueError, KeyError) as error:
        print(f"Error al leer los datos de ventas: {error}")

    except OSError as error:
        print(f"Error al abrir el archivo de ventas: {error}")

def revisar_stock(inventario):
    """
    Identifica los productos cuyo stock es igual o inferior al mínimo.

    Parámetros:
        inventario (list): Lista de diccionarios con los productos.

    Retorna:
        None: Muestra una alerta para los productos que necesitan reabastecimiento.
    """

    print("\n========== REVISION DE STOCK ==========")

    productos_bajos = []

    for producto in inventario:
        if producto["stock"] <= producto["stock_minimo"]:
            productos_bajos.append(producto)
```

```
    if len(productos_bajos) == 0:
        print("Todos los productos tienen existencias suficientes.")
        return

    print("\n¡ALERTA! PRODUCTOS QUE NECESITAN REABASTECIMIENTO")
    print("-" * 80)

    print(
        f"{'CODIGO':<10}"
        f"{'PRODUCTO':<28}"
        f"{'STOCK ACTUAL':>15}"
        f"{'STOCK MINIMO':>15}"
    )

    print("-" * 80)

    for producto in productos_bajos:
        print(
            f"{producto['codigo']:<10}"
            f"{producto['nombre']:<28}"
            f"{producto['stock']:>15}"
            f"{producto['stock_minimo']:>15}"
        )

    print("-" * 80)
    print(f"Productos que necesitan reabastecimiento: {len(productos_bajos)}")

def mostrar_inventario(inventario):
    """
    Muestra el inventario completo ordenado según el criterio seleccionado.

    Parámetros:
        inventario (list): Lista de diccionarios con los productos.

    Retorna:
        None: Muestra el inventario ordenado por nombre, precio o existencias.
    """

    print("\n========== MOSTRAR INVENTARIO ==========")
```

```
print("1. Ordenar por nombre")
print("2. Ordenar por precio: menor a mayor")
print("3. Ordenar por existencias: menor a mayor")

opcion_orden = input("Seleccione como desea ordenar: ").strip()

if opcion_orden == "1":
    productos_ordenados = sorted(
        inventario,
        key=lambda producto: producto["nombre"].casefold()
    )

elif opcion_orden == "2":
    productos_ordenados = sorted(
        inventario,
        key=lambda producto: producto["precio"]
    )

elif opcion_orden == "3":
    productos_ordenados = sorted(
        inventario,
        key=lambda producto: producto["stock"]
    )

else:
    print("Opcion de ordenamiento no valida.")
    return

print("\n" + "=" * 85)
print("              INVENTARIO DE PATRICK'S VIDEOGAMES")
print("=" * 85)

print(
    f"{'CODIGO':<10}"
    f"{'NOMBRE':<28}"
    f"{'CATEGORIA':<18}"
    f"{'PRECIO':>12}"
    f"{'STOCK':>9}"
)
```

```
    print("-" * 85)

    registrar_bitacora("Inventario consultado y ordenado")

    for producto in productos_ordenados:
        print(
            f"{producto['codigo']:<10}"
            f"{producto['nombre']:<28}"
            f"{producto['categoria']:<18}"
            f"Q{producto['precio']:>10,.2f}"
            f"{producto['stock']:>9}"
        )

    print("=" * 85)

def registrar_bitacora(mensaje):
    """
    Registra una operación en el archivo de bitácora con fecha y hora.

    Parámetros:
        mensaje (str): Descripción de la operación realizada.

    Retorna:
        None: Escribe la operación en el archivo de bitácora.
    """

    try:
        with open(ARCHIVO_BITACORA, "a", encoding="utf-8") as archivo:
            fecha_hora = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
            archivo.write(f"{fecha_hora} - {mensaje}\n")

    except OSError as error:
        print(f"Error al escribir en la bitacora: {error}")

def mostrar_menu():
    """
    Muestra las opciones disponibles del sistema de inventario y ventas.

    Parámetros:
        Ninguno.
```

```
    Retorna:
        None: Muestra el menú principal en la consola.
    """

    print("\n========================================")
    print("      PATRICK'S VIDEOGAMES")
    print("    SISTEMA DE INVENTARIO Y VENTAS")
    print("========================================")
    print("1. Agregar producto")
    print("2. Buscar producto")
    print("3. Registrar venta")
    print("4. Mostrar inventario")
    print("5. Reporte de ventas del dia")
    print("6. Revisar stock minimo")
    print("7. Reabastecer producto")
    print("8. Salir")
    print("=" * 40)
    print("========================================")

inventario = cargar_inventario()

if inventario is None:
    print("No se pudo cargar el inventario. El programa se cerrara para proteger los datos.")
    raise SystemExit

def reabastecer_producto(inventario):
    """
    Permite aumentar las existencias de un producto.

    Parámetros:
        inventario (list): Lista de productos.

    Retorna:
        None
    """
    codigo = input("Ingrese el código del producto: ").strip().upper()

    producto_encontrado = None
```

```
    for producto in inventario:
        if producto["codigo"] == codigo:
            producto_encontrado = producto
            break

    if producto_encontrado is None:
        print("No se encontró el producto.")
        return

    try:
        cantidad = int(input("Ingrese las unidades que desea agregar: "))

        if cantidad <= 0:
            print("La cantidad debe ser mayor que cero.")
            return

        producto_encontrado["stock"] += cantidad

        guardar_inventario(inventario)

        registrar_bitacora(
            f"Reabastecimiento: {codigo}, unidades agregadas: {cantidad}"
        )

        print("¡Existencias actualizadas correctamente!")
        print("Producto:", producto_encontrado["nombre"])
        print("Nuevo stock:", producto_encontrado["stock"])

    except ValueError:
        print("Error: ingrese una cantidad válida.")
while True:
    mostrar_menu()

    opcion = input("Seleccione una opcion: ").strip()

    if opcion == "1":
        agregar_producto(inventario)
        guardar_inventario(inventario)

    elif opcion == "2":
```

```
    buscar_producto(inventario)

elif opcion == "3":
    registrar_venta(inventario)

elif opcion == "4":
    mostrar_inventario(inventario)

elif opcion == "5":
    reporte_ventas_dia()

elif opcion == "6":
    revisar_stock(inventario)

elif opcion == "8":
    guardar_inventario(inventario)
    print("Gracias por usar Patrick's VideoGames.")
    break

elif opcion == "7":
    reabastecer_producto(inventario)

else:
    print("Opcion no valida. Intente nuevamente.")
```